



Introdução à Inteligência Artificial

Trabalho Prático 2

Problema de Otimização

David Reis nº 2022132864

João Rosa nº 2022131973

Índice

1. Introdução

O objetivo deste trabalho é implementar e testar métodos de otimização que encontrem soluções de boa qualidade para diferentes instâncias do problema a seguir descrito.

Dado um grafo e um valor inteiro k , este problema consiste em encontrar um subconjunto de vértices de tamanho k , tal que todos os vértices possuam pelo menos uma ligação e o custo das arestas dentro do subconjunto seja mínimo.

Formalmente o problema é definido:

Dados:

- um grafo não direcionado $G = (V, A)$, composto por um conjunto V de vértices ligados entre si por arestas pesadas A
- um inteiro k

Problema:

- encontrar um subconjunto de vértices S , de tamanho k , tal que $S \subseteq V$, com todos os vértices de S com pelo menos uma ligação, de forma a minimizar o custo total das arestas desse subconjunto
- o objetivo deste problema é, portanto, de minimização.

2. Desenvolvimento

Neste capítulo são apresentadas todas as propostas e alterações implementadas, assim como estratégias criadas para a simulação. São descritas algumas experiências realizadas e conclusões retiradas a partir das mesmas.

2.1. Exemplo dos ficheiros tratados

k 18	Valor de k
p edge 45 918	Grafo com 45 vértices e 918 arestas
e 2 1 4	Primeira aresta liga vértices 1 e 2 com custo 4
e 3 1 2	Segunda aresta liga vértices 1 e 3 com custo 2
e 3 2 5	
e 4 1 15	
e 4 2 10	
...	...
e 45 39 3	
e 45 40 5	
e 45 41 12	
e 45 42 10	Última aresta liga vértices 42 e 45 com custo 10

2.2. Funções

2.2.1 Função init_dados

A função “init_dados” lê as informações do ficheiro, guarda os dados sobre o k, o número de vértices e de arestas e guarda os valores de cada aresta numa matriz retornando um ponteiro para essa matriz.

```
int* init_dados(char *nome, int *v, int *ar, int *k)
{
    FILE *f;
    int *p;
    int i, j, v1, v2, valor;

    f=fopen( nome, "r");
    if(!f)
    {
        printf( "Erro no acesso ao ficheiro dos dados\n");
        exit( 1);
    }

    fscanf( f, " %d", k);
    // Numero de vertices
    fscanf( f, " \n edge %d %d", v, ar);
    // Alocacao dinamica da matriz
    p = malloc( sizeof(int) * (*v) * (*v));
    if(!p)
    {
        printf( "Erro na alocacao de memoria\n");
        exit( 1);
    }
}
```

```
for(i=0; i<*v; i++)
    for(j=0; j<*v; j++)
        p[i*(*v)+j]=0;

while(fscanf( f, " \ne %d %d %d", &v1, &v2, &valor) == 3){
    p[(v1-1)*(*v)+(v2-1)]=valor;
    p[(v2-1)*(*v)+(v1-1)]=valor;
}
fclose( f);
return p;
}
```

2.2.2 Função trepa_colinas

A função “trepa_colinas” é um algoritmo que começa com uma solução inicial, obtém o custo total dessa solução e depois, gera vizinhos, avalia cada vizinho e substitui a solução anterior pelo vizinho se o custo total do vizinho for menor que o da solução. O processo de criação e comparação do vizinho é repetido por um número de iterações “num_iter”. A função retorna o custo final da solução após a aplicação do algoritmo.

```
int trepa_colinas(int sol[], int *mat, int vert, int num_iter)
{
    int *nova_sol, custo, custo_viz, i;

    nova_sol = malloc( Size: sizeof(int)*vert);
    if(nova_sol == NULL)
    {
        printf( Format: "Erro na alocao de memoria");
        exit( Code: 1);
    }
    // Avalia solucao inicial
    custo = calcula_fit( a: sol, mat, vert);
    for(i=0; i<num_iter; i++)
    {
        gera_vizinho( a: sol, b: nova_sol, n: vert);
        // Avalia vizinho
        custo_viz = calcula_fit( a: nova_sol, mat, vert);
        if(custo_viz < custo)
        {
            substitui( a: sol, b: nova_sol, n: vert);
            custo = custo_viz;
        }
    }
    free( Memory: nova_sol);
    return custo;
}
```

2.2.3 Função gera_vizinho

A função “gera_vizinho”, como o nome indica, gera um vizinho da solução original trocando aleatoriamente alguns valores 0s por 1s e alguns valores 1s por 0s. Isso acontece “tam_viz” vezes.

```
void gera_vizinho(int a[], int b[], int n)
{
    int i, p1, p2, indice=2;

    for(i=0; i<n; i++){
        b[i]=a[i];
        for(i=0; i<indice; i++) {
            // Encontra posicao com valor 0
            do
            {
                p1 = random_l_h( min: 0, max: n - 1);
                while (b[p1] != 0);
            }
            // Encontra posicao com valor 1
            do
            {
                p2 = random_l_h( min: 0, max: n - 1);
                while (b[p2] != 1);
            }
            // Troca
            b[p1] = 1;
            b[p2] = 0;
        }
    }
}
```

2.2.4 Função calcula_fit

A função “calcula_fit”

verifica as conexões válidas entre os vértices marcados em a[], acumula o custo total, verifica se há conexões inválidas e se a solução for válida, retorna o custo total dividido por 2.

```
int calcula_fit(int a[], int *mat, int vert) {
    int total = 0;
    // Array de conexões
    int con[vert];
    for (int i = 0; i < vert; i++) {
        con[i] = 0;
    }

    // Loop por toda a matriz para verificar se é válida
    for (int i = 0; i < vert; i++) {
        // Verifica se a linha pertence à solução
        if (a[i] == 1) {
            for (int j = 0; j < vert; j++) {
                // Verifica se há uma conexão entre os vértices i e j

                if (a[j] == 1 && *(mat+i*vert+j) != 0) {
                    con[i] = 1;
                    con[j] = 1;
                    // Adiciona o custo
                    // printf("%d",*(mat + i * vert + j));
                    total += *(mat + i * vert + j);
                }
            }
        }
    }
}
```



```

// Verifica se a solução é válida
for (int i = 0; i < vert; i++) {
    if (con[i] == 0 && a[i] == 1) {
        // Solução inválida
        return 5000;
    }
}

// Retorna o custo total
return total/2;
}

```

2.2.5 Função gera_sol_inicial

A função “gera_sol_inicial” cria uma solução aleatória atribuindo valor 1 a k vértices escolhidos aleatoriamente entre os vértices totais. Os demais vértices têm valor 0 na solução.

```

void gera_sol_inicial(int *sol, int v, int k)
{
    int i, x;
    for(i=0; i<v; i++)
    {
        sol[i]=0;
    }
    for(i=0; i<k; i++)
    {
        do{
            x= random_l_h( min: 0, max: v-1);
        }while(sol[x]!=0);
        sol[x]=1;
    }
}

```

2.2.6 Função leAE

A função “leAE” é uma bastante idêntica á função “init_dados”, juntamente com obter os dados do ficheiro, também cria uma struct onde irá armazenar as probabilidades de mutação e recombinação tal como o tamanho do pop e o número de gerações, a função retorna a struct

```
struct info leAE(char *nome_fich){
    struct info temp;
    int v1,v2,valor,*p;
    FILE *f;

    f = fopen( Filename: nome_fich, Mode: "r");
    if(!f){
        printf( Format: "Erro ao abrir o ficheiro\n");
        exit( Code: 1);
    }

    fscanf( File: f, Format: "k %d", &temp.k);
    // Numero de vertices
    fscanf( File: f, Format: "\np edge %d %d", &temp.v,&temp.ar);
    p = malloc( Size: sizeof(int)*(temp.v)*(temp.v));
    while(fscanf( File: f, Format: "\ne %d %d %d",&v1,&v2,&valor)==3){
        p[(v1-1)*(temp.v)+(v2-1)]=valor;
        p[(v2-1)*(temp.v)+(v1-1)]=valor;
    }
}
```

```

fclose( File: f);
temp.pm = 0.01;
temp.pr = 0.3;
temp.tsize = 2;

temp.popsiz = 50000;
temp.numGeracoes = 100;

// printf("\nTamanho da populacao: ");scanf("%d",&temp.popsiz);
// printf("Tamanho do torneio: ");scanf("%d", &temp.tsize);
//printf("Numero de geracoes: ");scanf("%d", &temp.numGeracoes);
printf( Format: "\n");
return temp;
}

```

2.2.6 Função init_pop

A função “init_pop” cria uma população inicial para um algoritmo evolutivo, onde cada cromossomo é inicializado aleatoriamente com valores binários.

```

pchrom initPop(struct info *d) {
    pchrom pop;
    pop = malloc( Size: sizeof(chrom) * d->popsiz);

    for (int j = 0; j < d->popsiz; j++) {

        int aux[d->v];
        for (int i = 0; i < d->v; i++)
            aux[i] = 0;

        for (int i = 0; i < d->k; i++) {
            int x;
            do {
                x = random_l_h( min: 0, max: d->v - 1);
            } while (aux[x] != 0);
            aux[x] = 1;

            pop[j].p[x] = aux[x];
        }
    }

    return pop;
}

```

2.2.7 Função verificaGeral_2

A função “verificaGeral_2” é responsável por avaliar a aptidão de cada indivíduo na população

“pop” usando um algoritmo de avaliação individual.

```
void evaluate(pchrom pop, struct info d, int mat[][2])
{
    int i;

    for (i=0; i<d.popsiz; i++){
        reparacao(pop[i].p, d, mat);
        pop[i].fitness = eval_individual(sol: pop[i].p, d, mat, v: &pop[i].valido);
    }
}
```

2.2.8 Função avaliaIndividual_2

A função “avaliaIndividual_2” avalia e calcula o custo total de uma solução representada por um cromossomo, contando o numero de elementos com o valor 1 e verificando se esse número é igual a k, e verifica também as conexões entre os vértices, se a solução for válida retorna-se o custo total a dividir por 2, senão, retorna-se o valor 5000.

```

int avaliaIndividual_2(int solucao[], struct info *d, int *grafo, int *valido) {
    int total = 0, contador=0;
    // Array de conexões
    int con[d->v];
    for (int i = 0; i < d->v; i++) {
        con[i] = 0;
    }

    for(int y=0;y<d->v;y++)
    {
        if(solucao[y]==1)
        {
            contador++;
        }
    }
    if(contador!=d->k)
    {
        *valido=1;
    }
    else
    {
        *valido=0;
    }
}

```

```

    for (int i = 0; i < d->v; i++) {
        // Verifica se a linha pertence à solução

        if (solucao[i] == 1) {
            for (int j = 0; j < d->v; j++) {
                // Verifica se há uma conexão entre os vértices i e j
                if (solucao[j] == 1 && *(grafo + i * d->v + j) != 0) {
                    con[i] = 1;
                    con[j] = 1;
                    // Adiciona o custo

                    total += *(grafo + i * d->v + j);
                }
            }
        }
    }

    // Verifica se a solução é válida
    for (int i = 0; i < d->v; i++) {
        if ((con[i] == 0 && solucao[i] == 1) || *valido==1) {
            // Solução inválida
            return 5000;
        }
    }

    return total/2;
}

```

2.2.9 Função torneio_selecao

A função “torneio_selecao” implementa um processo de torneio, onde pares aleatórios de indivíduos são comparados e o de menor aptidão (fitness) é escolhido como pai para compor a nova população de pais.

```
void torneioSelecao(pchrom pop, struct info *d, pchrom pais){
    int a,b;

    for(int i = 0; i < d->popsiz; i++){

        //gero um indice aleatorio
        a = random_l_h( min: 0, max: (d->popsiz-1));

        // gero outro indice aleatorio ate ambos serem diferentes
        do
        |   b = random_l_h( min: 0, max: (d->popsiz-1));
        while(a == b);

        if(pop[a].fitness < pop[b].fitness)
        |   pais[i] = pop[a];
        else
        |   pais[i] = pop[b];
    }
}
```

2.2.10 Função recombinação

A função “recombinacao” implementa a recombinação entre pares de pais, onde um ponto de corte aleatório é escolhido e os genes dos filhos são formados pela combinação dos genes dos pais antes e depois desse ponto de corte. A probabilidade de ocorrer recombinação é controlada por “d->pr”. Se não houver recombinação, os filhos são idênticos aos pais.

```

void recombinaçao(pchrom pais, struct info *d, pchrom filhos){

    //ponto de corte aleatorio

    int ponto,j;

    for(int i = 0; i < d->popsiz; i+=2){

        // vai haver ponto de corte
        if(rand_01() < d->pr){
            ponto = random_l_h( min: 0, max: d->v);
            for(j = 0; j < ponto; j++){
                // passamos ao mesmo tempo os numeros ate ao ponto de corte
                filhos[i].p[j] = pais[i].p[j];
                filhos[i+1].p[j] = pais[i+1].p[j];
            }
            // preenchemos o primeiro filho
            if(j == ponto){
                for(int x = j; x < d->v; x++)
                    filhos[i].p[x] = pais[i+1].p[x];
            }
            // preenchemos o segundo filho
            j = ponto;

```

```

                for(int x = j; x < d->v; x++)
                    filhos[i+1].p[x] = pais[i].p[x];
            }

            else{
                // faz uma copia exata porque nao houve a pr
                filhos[i] = pais[i];
                filhos[i+1] = pais[i+1];
            }
        }
    }
}

```

2.2.11 Função mutacao

A função “mutacao” realiza a mutação nos cromossomos da população de filhos. Para cada gene de cada filho, verifica-se se uma mutação deve ocorrer com base na probabilidade “d->pm”. Se a mutação for acionada, um novo valor aleatório, diferente do valor atual do gene, é escolhido para substituir o gene atual. Isso ajuda a introduzir variabilidade genética na população e contribui para a exploração do espaço de busca do algoritmo genético.

```
void mutacao(pchrom filhos, struct info *d){
    int num;

    for(int i = 0; i < d->popsiz; i++){
        for(int j = 0; j < d->v; j++){
            if(rand_01() < d->pm){

                // encontramos um nr aleatorio != do nr atual
                do{
                    num = random_l_h( min: 0, max: 1);
                }while(num == filhos[i].p[j]);

                // alteramos o nr atual pelo num
                filhos[i].p[j] = num;
            }
        }
    }
}
```


2.3. Testes

Trepas Colinas com Vizinhança 1					
NUM_VERT		1000it	10000it	50000it	100000it
file1.txt	Melhor	45	45	48	48
	MBF	59.3	55.7	69.5	55.6
file2.txt	Melhor	15	14	14	13
	MBF	27.8	21	29.7	22.9
file3.txt	Melhor	352	336	336	336
	MBF	396.4	364.6	367.4	368.3
file4.txt	Melhor	inválido	inválido	8	inválido
	MBF	inválido	inválido	3502.5	inválido
file5.txt	Melhor	inválido	inválido	inválido	inválido
	MBF	inválido	inválido	inválido	inválido

Trepas Colinas com Vizinhança 1 e aceitando custo inicial

NUM_VERT		1000it	10000it	50000it	100000it
file1.txt	Melhor	45	45	50	52
	MBF	60.2	59.5	59.8	55.4
file2.txt	Melhor	15	15	15	15
	MBF	23.3	26.7	26.6	20.6
file3.txt	Melhor	340	336	336	336
	MBF	387.9	374.1	362.4	387.3
file4.txt	Melhor	10	9	7	9
	MBF	22.5	11	8.8	9.5
file5.txt	Melhor	inválido	inválido	14	12
	MBF	inválido	inválido	3007.1	1014.8

Nestes 2 testes usamos o algoritmo trepa-colinas, e de um para o outro apenas alteramos o algoritmo para que este aceitasse as soluções iniciais.

Trepa Colinas com Vizinhança 2

NUM_VERT		1000it	10000it	50000it	100000it
file1.txt	Melhor	52	45	45	45
	MBF	58.5	52.8	52.5	48.7
file2.txt	Melhor	15	8	11	11
	MBF	30.8	19	18.4	15.5
file3.txt	Melhor	399	336	336	336
	MBF	419	366.9	363	358.7
file4.txt	Melhor	31	12	7	7
	MBF	3510.4	16.8	2004.9	3003.1
file5.txt	Melhor	inválido	inválido	29	inválido
	MBF	inválido	inválido	4502.9	inválido

Trepa Colinas com Vizinhança 2 e aceitando custo inicial

NUM_VERT		1000it	10000it	50000it	100000it
file1.txt	Melhor	48	45	45	45
	MBF	60.3	52.4	49.3	51.5
file2.txt	Melhor	17	15	8	10
	MBF	38.1	19.4	14.7	16.1
file3.txt	Melhor	374	336	336	336
	MBF	430.6	386.8	366.4	374.5
file4.txt	Melhor	36	12	7	7
	MBF	54.7	16	8.3	10.6
file5.txt	Melhor	inválido	83	27	11
	MBF	inválido	3066.7	2515	27

Nestas duas tabelas já alteramos o valor da vizinhança para 2 e na primeira não aceitávamos o custo inicial, no entanto na segunda já o aceitamos.

Evolutivo com 100 gerações pm=0.01 pr=0.3

NUM_VERT		1000ps	10000ps	25000ps	50000ps
file1.txt	Melhor	48	45	45	45
	MBF	58.7	50.1	46.7	45.8
file2.txt	Melhor	36	38	18	16
	MBF	52.3	57.4	22.3	27.5
file3.txt	Melhor	554	529	498	480
	MBF	589.9	554.6	505.5	505.2
file4.txt	Melhor	77	58	49	34
	MBF	97.9	71.2	63.3	52.1
file5.txt	Melhor	inválido	inválido	324	244
	MBF	inválido	inválido	4071.9	3124.6

Evolutivo com 100 gerações pm=0.05 pr=0.3

NUM_VERT		1000it	10000it	50000it	100000it
file1.txt	Melhor	45	45	50	50
	MBF	51.9	45.9	61.5	58.8
file2.txt	Melhor	14	8	25	24
	MBF	19.4	7.9	37.5	32.3
file3.txt	Melhor	539	515	512	496
	MBF	564.8	544.2	529.9	519.4
file4.txt	Melhor	57	47	48	51
	MBF	99.8	69.6	62.8	62.1
file5.txt	Melhor	inválido	654	320	264
	MBF	inválido	4565.4	3604.7	1270.7

Para estas duas tabelas já utilizamos o algoritmo evolutivo, com as mudanças que estão assinaladas nas imagens e reparamos que as soluções a partir do file3.txt já não são tão rigorosas como no algoritmo trepa-colinas.

2.4. Conclusão

Uma das conclusões é que o trepa colinas conseguiu apenas os melhores resultados em ficheiros mais pequenos, quando era deparado com um grande número de vértices a sua eficácia desce consideravelmente.

Outra conclusão foi ao baixarmos o valor de pm as soluções em ficheiros mais pequenos foram mais constantes e estáveis, porém, em ficheiros mais “pesados”, os resultados foram mais elevados do que o esperado.